G1Nav: Engineering Report

Humanoid locomotion RL with hierarchical target navigation for the Unitree G1 in MuJoCo / MJX: an English instruction selects a target object (by keyword matching in the recorded demos), a Navigator turns its position into a velocity command, and a PPO policy trained from scratch outputs 29 lower-body PD joint targets at 50 Hz. All motion is physics-simulated; there is no kinematic playback.

> **Scope.** In the recorded demo runs the target label is chosen by keyword overlap between the instruction and the scene's object labels (`code/navigation/grounding.py`, called from `cortex_eagle.py`), and the target position is read from the generated scene (simulator ground truth, `navigate.py`). The GR00T N1.6 Eagle encoder is run on the start ego frame and the shape/norm of its embedding are logged, but the embedding does not influence the chosen target. The walking policy observes proprioception and the velocity command only; no camera image enters the control loop. Where this report describes a vision-language grounding stage or the colour/shape + ego-depth localisation path, it describes an intended design, not something the demos show.

Every number in this report is checked against files in the repository by `python analysis/verify_claims.py`; the claim-by-claim audit is in claims.md.

---

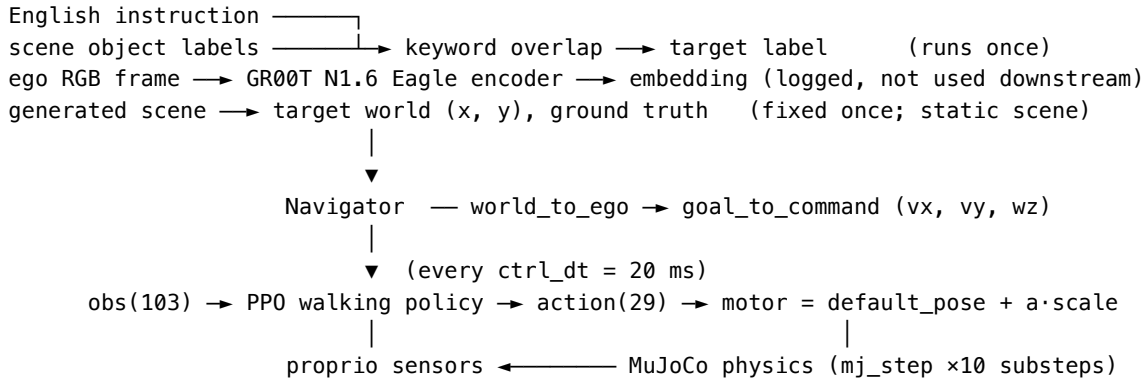
1. Summary

- **Target selection (no training).** The instructed target is the scene object whose label words overlap most with the instruction. Its world position is taken from the generated scene. The Eagle vision encoder from the Isaac GR00T N1.6 checkpoint is run on the ego frame and its embedding statistics are recorded; it is a placeholder for a learned grounding stage.
- **Control (learned).** A velocity-conditioned PPO walking policy is trained once in MuJoCo Playground's `G1JoystickFlatTerrain` (MJX) and reused for every instruction. Its action is a lower-body PD joint-position target.
- **Navigator (hand-written).** A turn-then-go controller converts the target world coordinate and the robot's odometry into a velocity command `(vx, vy, wz)` at every control step.

Only the control half is learned and evaluated here. The grounding stage is keyword matching with a ground-truth position, so this project says nothing about whether a vision-language model can do the grounding.

---

2. System architecture



- **Slow stage (once per episode):** target selection. The scene is static, so the target's world coordinate is fixed once and tracked with odometry.
- **Fast stage (every step):** the PPO policy maps the 103-D observation to 29 joint targets at 50 Hz.

Observation (103-D, matches Playground's G1 Joystick `_get_obs`)

`[ local_linvel_pelvis(3), gyro_pelvis(3), gravity in pelvis-IMU frame(3), command(3), qpos[7:]-default_pose(29), qvel[6:] (29), last_action(29), cos/sin gait-phase(2) ]`

The linear velocity and gyro terms are read from the named sensors the training environment uses (`local_linvel_pelvis`, `gyro_pelvis`); the gravity term is `site_xmat[imu_in_pelvis].T @ [0, 0, −1]`, exactly as in training (see §5.7).

Action (29-D joint targets)

`motor_target = default_pose + action · action_scale (0.5)`. The environment applies these as PD position targets, so the network output is the joint-target vector itself.

Policy network (read from the checkpoint)

Policy MLP 103 → 512 → 256 → 128 → 58 (mean and scale for 29 actions); value MLP 216 → 512 → 256 → 128 → 1 (asymmetric critic on the 216-D privileged state); a running observation normaliser is stored as `params[0]`.

---

3. Design rationale (intended design; not tested here)

The project was built around three assumptions. They motivated the structure; none of them is evaluated in this repository.

1. **Separate the slow "which object, where" decision from fast balance and stepping.** The first can run once per episode for a static scene; the second must run at the control rate.
2. **Keep a pretrained vision-language backbone frozen** rather than fine-tune it on a small task-specific dataset. In the current code the backbone (GR00T N1.6 Eagle) is run but not used for the decision.
3. **Learn only what needs learning.** Locomotion is learned from simulation reward with PPO; no demonstration data is collected. Target selection is hand-written.

3.1 Data: there is no collected dataset

No trajectories were collected and nothing is trained on a dataset:

- **Target selection** is keyword matching against the scene's object labels; the Eagle encoder is run without training and its embedding is only logged.
- **Control** is trained by on-policy PPO inside MJX; the only data are the transitions that the 8192 parallel simulators generate and consume during training.

Instructions are generated from seeded templates (`code/envs/instructions.py`) in three tiers (go-to / follow / turn-after-pass); scenes come from `code/envs/scene_gen.py` (seed → exact 3-object arena). The recorded demos use only the go-to tier on seeds 0–2. See `dataset/NOTE.md`.

3.2 Target selection components

- `grounding.py`: the keyword rule. The score of a label is the number of its words that occur in the lower-cased instruction; the highest score wins, ties go to the first label.
- `cortex_eagle.py`: runs the Eagle encoder from the GR00T N1.6 checkpoint on the ego frame, records the embedding's shape and norm, then calls `grounding.keyword_target`. Its JSON output keeps the historical key name `vlm_target` for the keyword-selected label.
- `navigate.py`: reads the chosen target's `(x, y)` from the generated scene. Its `--use_vlm` flag is declared but not implemented.
- `color_detect.py`: HSV colour masking and shape heuristics. Together with ego-depth and camera geometry this would give a bearing/distance estimate (`Navigator.lock_goal_from_ego`); it is **not used** in the recorded runs and was not evaluated.

Recorded: the Eagle encoder ran on each start frame (286 tokens × 2048-d embedding, logged in `videos/*.cognition.json`), and the keyword rule picked the instructed object in all three demo scenes. Each scene's instruction names a unique colour + shape combination, so this is a check of the string rule, not of perception.

---

4. Control: the PPO walking policy

- **Environment:** MuJoCo Playground `G1JoystickFlatTerrain` (MJX). **Algorithm:** brax PPO, 8192 parallel environments, asymmetric critic, Playground's tuned hyperparameters.
- **Domain randomisation:** `registry.get_domain_randomizer` plus a separate `eval_env`, as in the official locomotion recipe. Without it the reward rose but the policy did not walk.
- **Reward:** the Playground G1 Joystick reward with its default scales in the final run (`--termination -100 --track_lin 1.0 --stand_still -1.0`, which are the defaults; §5.6 explains why earlier edits were reverted).
- **Checkpoint:** `checkpoint/walk_t4_latest.pkl` stores `{"step": 154828800, "params": ...}` and is byte-identical to `walk_t4_step0154828800.pkl`.

Walking emergence

Rewards are the training-time evaluation rewards in `checkpoint/walk_t4_metrics.jsonl`. Forward velocity, net distance and minimum torso height are a CPU re-evaluation of every committed checkpoint with `code/locomotion/walk_eval.py` (raw environment, no auto-reset, command vx = 0.6 m/s, 250 steps = 5 s), stored in `results/walk_sweep_cpu/walk_sweep.json`.

step	eval reward	fwd_vel (cmd 0.6)	net dist	min torso z	verdict
0	−6.30	+0.36	0.45 m	−0.78	falls
10.3 M	−2.83	+0.28	0.55 m	−0.78	falls
31.0 M	−2.09	−0.27	0.76 m	−0.76	falls
41.3 M	**+0.03**	**+0.32**	**1.54 m**	**0.73**	**first checkpoint that walks**
61.9 M	+3.99	+0.50	2.51 m	0.73	walks
144.5 M	+8.75	+0.58	2.89 m	0.74	walks (highest fwd_vel in the sweep)
154.8 M (final)	+9.29	+0.52	2.59 m	0.74	walks

The `min_z` field is `qpos[2]` as reported by `walk_eval.py`; negative values occur only for checkpoints that fall. The training-time rollout of the final checkpoint (`checkpoint/walk_t4_rollout.json`, 200 steps = 4 s, same command) reports 0.519 m/s mean local forward velocity, 2.07 m net displacement and 0.745 m minimum torso height.

Throughput, from the `elapsed_min` field of `walk_t4_metrics.jsonl`: about 2.3 M steps/min between the first and last evaluation (10.3 M → 154.8 M steps in 61.9 min), about 2.0 M steps/min over the whole 77-minute run including compilation. The GPU (an RTX 6000 Ada) is not recorded in the committed files.

Closed-loop target reaching

Three seeded scenes, target label chosen by keyword match, target position from the generated scene. "Reached" means the Navigator's stop condition fired (planar distance ≤ 0.55 m, `REACH_RADIUS` in `action_module.py`) without the torso dropping below 0.4 m; the final distance is measured after the 2 s standing post-roll.

seed	instruction	chosen target	reached	final dist	torso z	video frames
0	"go to the orange cylinder"	orange cylinder	yes	0.39 m	0.75 m	630
1	"go to the red cube"	red cube	yes	0.41 m	0.76 m	387
2	"go to the purple cylinder"	purple cylinder	yes	0.54 m	0.75 m	668

The recorded GPU runs' `NAV` log lines are not in the repository. Re-running the same three episodes on CPU (`scripts/rerun_demo_episodes.py`, `results/demo_rerun_cpu/`) reproduces the final distances and torso heights to the printed precision; frame counts match for seeds 0 and 2 and differ by one frame for seed 1 (386 vs 387). The seed-2 final distance (0.54 m) is just inside the 0.55 m stop radius.

Three things went wrong on the way (§5): the marching-in-place reward/auto-reset illusion, falls on every yaw from a wrong-frame gravity term, and toppling from unnormalised observations.

---

5. What went wrong, and the fixes (engineering log)

1. **cuSolver INTERNAL error (A6000).** brax PPO reset/eval crashed on an Ampere GPU with jax 0.5.3. Moving to an Ada-generation GPU (RTX 6000 Ada) removed it. The pinned jax 0.5.3 / CUDA 12.6 stack also has no kernels for Blackwell (sm_120).
2. **Reward rises but the robot does not walk.** Root cause: missing domain randomisation. Adding `randomization_fn` and `eval_env` made walking learnable.
3. **Observation mismatch.** Building the 103-D observation by hand (from qpos/qvel/quaternion) made a walking policy fall. Fix: read the named sensors the environment reads.
4. **Velocity-command frame.** `tracking_lin_vel` tracks local (heading-frame) pelvis velocity, so a forward command moves the robot along its heading, not world-X. Early evaluations that measured world-X under-reported real motion.
5. **Throughput trap.** Rendering a rollout video at every evaluation with `num_evals=1000` throttled training about 50× (0.2 M steps/min). Fix: `num_evals=50` (an evaluation every ~10 M steps).
6. **The "marches in place" illusion and the auto-reset mask.** With a softened `termination=-3`, the policy converged to a high-reward (+19) behaviour that looked like walking but did not translate. Two compounding causes:
   - **Reward:** with falling nearly free, marching in place safely collects `feet_air_time`, `feet_phase`, posture and angular-tracking reward without forward motion.
   - **Measurement:** the in-training rollout ran inside `wrap_for_brax_training`, whose `BraxAutoResetWrapper` resets the robot to the start pose on every fall. A falling policy therefore rendered as stable stepping (min_z ≈ 0.74, x ≈ 0). The raw environment showed that it falls.

   Fix: restore the default `termination=-100` and run every evaluation on the raw environment (no auto-reset), measuring local forward velocity, net horizontal distance and min_z. Walking first appears at the 41.3 M checkpoint (§4).
7. **The closed loop fell over: two inference-time observation bugs.** The policy walked in `walk_eval` but toppled within ~1.6 s in the navigation scene. Found by comparing the observation vector term by term with the training environment's `_get_obs`:
   - **(a) Missing observation normalisation.** Training used `normalize_observations=True`, so the checkpoint stores a `RunningStatisticsState` as `params[0]`. Inference built the network without `preprocess_observations_fn`, so the policy saw raw observations. Fix: build the network with `preprocess_observations_fn=running_statistics.normalize` in `navigate.py` and `walk_eval.py`. This alone let the robot reach a straight-ahead target.
   - **(b) Gravity in the wrong frame.** Training's projected-gravity term is `site_xmat[imu_in_pelvis].T @ [0,0,-1]` (world down vector in the pelvis-IMU frame). The deployed code used `-upvector_pelvis`, which coincides with it only while the robot walks straight; on every yaw the two diverge and the policy misreads its tilt. This matched the symptom (straight = fine, every turn = fall at ~1.6 s). Fix: compute the term from the site matrix as in training.

   With both fixes, all three demo scenes are reached upright (§4).

Items 1, 5, 6 and 7 describe intermediate runs and code versions whose logs are not in the repository; they are the author's engineering record, not re-checkable results.

---

6. Reproduction

```
# environment (Ada GPU, fresh machine) with the pinned stack:
#   jax[cuda12]==0.5.3, brax==0.14.2, flax==0.10.6, mujoco==3.4.0, mujoco-mjx==3.4.0, playground==0.1.0
bash scripts/pod/setup_ada.sh

# train the walking policy (default reward scales)
```

```
python code/locomotion/train_walk.py --timesteps 500000000 --num_envs 8192 \
    --termination -100 --track_lin 1.0 --stand_still -1.0 --num_evals 50 \
    --out checkpoint/walk_t4

# raw-environment evaluation of one checkpoint, or a sweep over all of them
G1NAV_OUT_DIR=results python code/locomotion/walk_eval.py checkpoint/walk_t4_latest.pkl
G1NAV_OUT_DIR=results python code/locomotion/walk_eval.py --sweep checkpoint 'walk_t4_step*.pkl'

# seeded scene generation
python code/envs/scene_gen.py --seed 0

# keyword target selection + Eagle embedding log (needs the encoder at $G1NAV_EAGLE)
python code/navigation/cortex_eagle.py --image ego.png \
    --instruction "go to the orange cylinder" \
    --labels "orange cylinder,yellow ball,purple cube" --result cortex.json

# closed loop: target position from the scene → Navigator → walking policy → physics → video
python code/navigation/navigate.py --ckpt checkpoint/walk_t4_latest.pkl --seed 0 \
    --target "orange cylinder" --seconds 12 --out outputs/nav_seed0.mp4

# the three demo episodes, re-measured (CPU is enough)
python scripts/rerun_demo_episodes.py --cpu --outdir results/demo_rerun_cpu
```

The committed checkpoints end at 154.8 M steps. Their spacing (10,321,920 steps, i.e. 50 evaluations over about 500 M steps) is consistent with the 500 M-step command above being stopped early; the training log that would show this directly is not in the repository.

---

7. Limitations

- Target selection is keyword matching and the target position is ground truth; instructions must name an object's colour and shape. Nothing is learned or evaluated on the perception side.
- Three seeded scenes, one run each, static objects, flat terrain. No success rate over a larger scene distribution is reported.
- Static-scene assumption: the target is fixed once and tracked by odometry.
- The walking policy follows the MuJoCo Playground G1 Joystick recipe and reward. The work here is the coupling (Navigator, observation construction for deployment in a different scene) and the debugging needed to get forward walking out of MJX training on a single GPU (§5).
- Training ran in MJX; the closed-loop demos run the same model in plain MuJoCo with the training solver options (`navigate.py`). The demos are rendered from the physics that is being controlled.